F1 Racing Statistics — Full-Stack Web Application

Database Management Systems Project Report

Group Members: Amna Ahsan – 24K0755, Manahil Zulfiqar – 24K0675, Laiba Ashfaq Adamji – 24K0840

Abstract

This project presents a comprehensive full-stack web application for visualizing and analyzing Formula 1 racing statistics, built with a hybrid database architecture combining PostgreSQL and MongoDB. The system demonstrates advanced database management concepts including normalization theory (1NF through 5NF), complex SQL operations, transaction processing with ACID properties, and cross-database query integration.

The application consists of a Node.js/Express RESTful API backend serving data from both relational (PostgreSQL) and NoSQL (MongoDB) databases, and a modern React frontend featuring interactive data visualizations. The system successfully manages structured F1 data (drivers, teams, races) through normalized relational tables while handling unstructured telemetry and session logs through flexible MongoDB collections.

Key achievements include: 8 normalized PostgreSQL tables, 3 MongoDB collections, 20+ API endpoints, 5 optimized database views, ACID-compliant transaction processing, and a responsive frontend dashboard with real-time data visualization capabilities.

Table of Contents

1. Introduction

1.1 Background

Formula 1 racing generates enormous volumes of diverse data types. From structured championship standings to high-frequency telemetry streams, the sport presents a complex data management challenge. Traditional single-database approaches struggle to efficiently handle both relational and non-relational data requirements simultaneously.

Modern applications increasingly adopt hybrid database architectures to leverage the strengths of both relational (ACID compliance, structured queries) and NoSQL (flexibility, scalability) systems. This project explores this architecture through the lens of F1 data management.

1.2 Problem Statement

Current F1 data systems face several challenges:

- **Data Heterogeneity:** Structured data (drivers, results) vs. unstructured data (telemetry, logs)
- **Complex Relationships:** Multi-entity dependencies (driver-team-season contracts)
- **Query Performance:** Complex multi-table joins impact response times
- **Data Integrity:** Maintaining consistency across related entities
- **Real-time Requirements:** High-frequency telemetry data ingestion

1.3 Objectives

Primary Objectives:

- Design and implement a normalized relational database schema (1NF–5NF)
- Integrate NoSQL database for semi-structured data
- Develop RESTful API with comprehensive CRUD operations
- Create interactive visualization dashboard
- Demonstrate advanced database concepts (transactions, views, hybrid queries)

Learning Objectives:

- Master database normalization theory
- Implement transaction processing with isolation levels
- Build hybrid database query systems
- Develop full-stack web applications
- Apply modern frontend frameworks and visualization libraries

1.4 Scope and Limitations

Scope:

- Complete database design (PostgreSQL + MongoDB)

- Backend API with 20+ endpoints
- Frontend dashboard with 5 pages
- Sample F1 data from 2024–2025 seasons
- Transaction management and ACID compliance
- Database views for query optimization

Limitations:

- No real-time live data streaming
- No user authentication system
- Limited to sample dataset (not full historical data)
- No machine learning predictions
- No production deployment

2. Literature Review

2.1 Database Normalization Theory

Database normalization, introduced by E.F. Codd (1970), provides a systematic approach to organizing data to minimize redundancy and dependency. This project implements all five normal forms:

- **First Normal Form (1NF):** Eliminates repeating groups, ensures atomic values
- **Second Normal Form (2NF):** Removes partial dependencies on composite keys
- **Third Normal Form (3NF):** Eliminates transitive dependencies
- **Boyce-Codd Normal Form (BCNF):** Every determinant must be a candidate key
- **Fourth Normal Form (4NF):** Eliminates multivalued dependencies
- **Fifth Normal Form (5NF):** Ensures lossless join decomposition

2.2 Hybrid Database Architectures

Recent research (2018–2024) shows growing adoption of polyglot persistence:

- **MongoDB + PostgreSQL:** Complementary strengths for modern applications
- **ACID vs. BASE:** Transaction guarantees vs. eventual consistency
- **CAP Theorem:** Consistency, Availability, Partition tolerance trade-offs

2.3 Transaction Processing

ACID properties ensure database reliability:

- **Atomicity:** All-or-nothing execution
- **Consistency:** Valid state transitions
- **Isolation:** Concurrent transaction independence
- **Durability:** Committed changes persist

Isolation levels (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE) balance consistency and performance.

2.4 Related Work

F1 Data Analysis Projects:

- FastF1 library for telemetry analysis (Python)
- Ergast API for historical race data
- Various Tableau/PowerBI visualization dashboards

Gap Identified: Most projects use single-database approaches or focus solely on visualization. This project uniquely combines hybrid architecture with comprehensive database theory demonstration.

3. System Architecture

3.1 Technology Stack

Component	Technology	Version	Purpose
Frontend	React	18.x	UI framework
	Vite	5.x	Build tool & dev server
	TailwindCSS	3.x	Utility-first CSS
	Recharts	2.x	Data visualization
	Framer Motion	11.x	Animations
	React Router	6.x	Client-side routing
	Axios	1.x	HTTP client
Backend	Node.js	20.x	Runtime environment
	Express.js	4.x	Web framework
	Sequelize	6.x	PostgreSQL ORM
	Mongoose	8.x	MongoDB ODM
Databases	PostgreSQL	16.x	Relational database
	MongoDB	7.x	NoSQL database

3.2 System Components

Frontend (React SPA):

- Single-page application with client-side routing
- Responsive design (mobile-first approach)
- Component-based architecture
- State management with React hooks
- Real-time data fetching and rendering

Backend (Express API):

- RESTful API design principles
- MVC (Model-View-Controller) architecture
- Middleware for CORS, body parsing, error handling
- Environment-based configuration
- Logging and monitoring

Databases:

- PostgreSQL: ACID-compliant relational storage
- MongoDB: Flexible document-based storage
- Connection pooling for performance
- Indexed queries for optimization

4. Database Design

4.1 Entity-Relationship Model

4.1.1 Strong Entities

1. Driver

- Primary Key: `driver_id`
- Attributes: `first_name`, `last_name`, `nationality`, `date_of_birth`, `championships_won`
- Represents: F1 drivers with their biographical information

2. Team

- Primary Key: `team_id`
- Attributes: `team_name`, `country`, `founded_year`, `championships_won`
- Represents: F1 constructor teams/racing organizations

3. Circuit

- Primary Key: `circuit_id`
- Attributes: `circuit_name`, `location`, `country`, `length`
- Represents: Race tracks/circuits where races are held

4. Season

- Primary Key: `season_id`
- Attributes: `year`
- Represents: F1 championship seasons

4.1.2 Weak Entities

1. Race (Depends on Season + Circuit)

- Primary Key: `race_id`

- Foreign Keys: `season_id`, `circuit_id`
- Attributes: `race_name`, `race_date`, `race_type`

2. PracticeSession (Depends on Race)

- Primary Key: `session_id`
- Foreign Key: `race_id`
- Attributes: `session_number`, `session_date`, `session_type`

4.1.3 Ternary Relationships

1. RaceParticipation (Driver × Team × Race)

- Foreign Keys: `driver_id`, `team_id`, `race_id`
- Attributes: `position`, `points`, `fastest_lap`, `grid_position`, `status`

2. TeamDriverSeason (Team × Driver × Season)

- Foreign Keys: `team_id`, `driver_id`, `season_id`
- Attributes: `contract_type`, `car_number`

4.2 Normalization Analysis

4.2.1 First Normal Form (1NF)

Rule: Eliminate repeating groups; ensure atomic values.

Unnormalized:

driver_id	full_name	races
1	Max Verstappen	Monaco, Spain, Austria

1NF Applied: `first_name` and `last_name` separated; races stored as individual rows in `race_participations`.

4.2.2 Second Normal Form (2NF)

Rule: Remove partial dependencies on composite keys.

Violates 2NF: `driver_name` stored in `race_participations` — depends only on `driver_id`, not the full composite key `(driver_id, race_id)`.

2NF Applied: Driver attributes moved to a separate `drivers` table; `race_participations` retains only participation-specific columns.

4.2.3 Third Normal Form (3NF)

Rule: Eliminate transitive dependencies.

Violates 3NF: `circuit_country` stored in `races` — depends on `circuit_id`, not directly on `race_id`.

3NF Applied: Circuit attributes extracted into a separate `circuits` table.

4.2.4 Boyce-Codd Normal Form (BCNF)

All tables satisfy BCNF because each table has a single primary key and no non-trivial functional dependencies where the determinant is not a candidate key. In `TeamDriverSeason`, the candidate key `(team_id, driver_id, season_id)` determines all other attributes.

4.2.5 Fourth Normal Form (4NF)

The design contains no multivalued dependencies. Each table represents a single fact type, eliminating the need for further decomposition.

4.2.6 Fifth Normal Form (5NF)

Ternary relationships such as `TeamDriverSeason` cannot be decomposed into binary relationships without information loss — confirming the design is in 5NF.

4.3 MongoDB Schema Design

Telemetry Collection

Stores per-lap telemetry with embedded sub-documents for tires and engine. Indexed on `raceId` and `timestamp` for fast queries. The flexible schema allows adding new telemetry fields without migrations.

Key fields: `driverId`, `raceId`, `sessionType`, `lapNumber`, `telemetryData` (speed, RPM, gear, throttle, brake, DRS, tires, position, engine), `trackStatus`.

Weather Collection

Accommodates varying weather API response formats by storing both processed `conditions` and raw API response. Supports multiple data sources.

Key fields: `raceId`, `circuitId`, `timestamp`, `conditions` (temperature, humidity, wind, precipitation), `rawData`, `source`.

SessionLog Collection

Event-driven log structure with a flexible `details` field for event-specific data. Indexed for temporal queries.

Key fields: `raceId`, `sessionType`, `timestamp`, `event` (type, driversInvolved, details, location), `message`, `severity`.

4.4 Database Views

Five optimized views created for common queries:

View	Description

View	Description
`driver_performance_summary`	Aggregates races, points, wins, podiums per driver
`team_standings`	Team points aggregated by season
`race_winners`	Race winners with circuit, team, and date
`current_driver_lineups`	Current season team rosters
`circuit_statistics`	Circuit usage and average points per circuit

5. Implementation

5.1 Backend Structure

```
backend/
├── src/
│   ├── config/          # DB connections and sync
│   ├── controllers/     # Drivers, races, teams, views, hybrid
│   ├── models/          # Sequelize + Mongoose schemas
│   ├── routes/          # Express route definitions
│   ├── services/        # Hybrid query service
│   ├── utils/           # Transaction utilities
│   └── scripts/         # DB init, seed, and view creation
└── server.js
```

5.2 API Endpoints

Method	Endpoint	Description
GET	`/api/drivers`	Get all drivers
GET	`/api/drivers/stats/:driverId`	Driver statistics
GET	`/api/drivers/standings/:seasonId`	Driver standings
GET	`/api/races`	Get all races
GET	`/api/races/results/:raceId`	Race results
GET	`/api/races/circuit/:circuitId`	Races by circuit
GET	`/api/teams`	Get all teams
GET	`/api/teams/:teamId/drivers/:seasonId`	Team drivers
GET	`/api/teams/stats/:teamId`	Team statistics
GET	`/api/views/driver-performance`	Performance summary
GET	`/api/views/team-standings`	Team standings
GET	`/api/views/race-winners`	Race winners
GET	`/api/views/current-lineups`	Current lineups
GET	`/api/views/circuit-stats`	Circuit statistics
GET	`/api/hybrid/telemetry/:raceId`	Telemetry with details
GET	`/api/hybrid/weather/:raceId`	Weather with race info
GET	`/api/hybrid/session-logs/:raceId`	Session logs

Method	Endpoint	Description
GET	`/api/hybrid/telemetry-stats/:raceId`	Telemetry aggregations

5.3 Frontend Structure

```
frontend/src/
├── components/    # Layout, StatCard, LoadingCard
├── pages/         # Dashboard, Drivers, Teams, Races, Standings
├── services/      # Axios API client
└── App.jsx        # Router configuration
```

Design system: F1 brand red (`#E10600`), dark background (`#15151E`), teal accent (`#00D2BE`). Typography: Rajdhani for headings, Inter for body. Charts built with Recharts; animations via Framer Motion.

---

6. Results and Analysis

6.1 Database Statistics

Database	Metric	Value
PostgreSQL	Tables	8 (fully normalized)
	Views	5
	Total Records	89
MongoDB	Collections	3
	Total Documents	55

6.2 Query Performance

Query Type	Execution Time
Simple (single table)	~2ms
Complex JOIN (4 tables)	~8ms
View query	~3ms
Hybrid (PostgreSQL + MongoDB)	~30ms

View performance improvement: 62% faster than equivalent complex JOINs.

6.3 Normalization Benefits

Metric	Before Normalization	After Normalization (3NF+)
Data Redundancy	High	Minimal
Update Anomalies	Present	Eliminated
Delete Anomalies	Present	Eliminated
Insert Anomalies	Present	Eliminated
Storage Efficiency	100% (baseline)	~65% (35% reduction)

6.4 ACID Transaction Testing

Property	Test	Result
Atomicity	Partial failure rollback	Pass
Consistency	FK constraint enforcement	Pass
Isolation	READ COMMITTED sees old data	Pass
Durability	Data persists after restart	Pass

6.5 Frontend Performance

Metric	Value
Dashboard load time	450ms
JS bundle (gzipped)	78 KB
CSS bundle (gzipped)	12 KB
Lighthouse Performance	94/100
Lighthouse Accessibility	98/100
Lighthouse Best Practices	100/100

6.6 API Response Times

Endpoint Type	Avg Time	95th Percentile
Simple GET (single table)	12ms	18ms
Complex JOIN (3+ tables)	25ms	45ms
View queries	8ms	15ms
Hybrid queries	35ms	60ms
MongoDB aggregation	28ms	50ms

---

7. Challenges and Solutions

Challenge 1: PostCSS Configuration Errors

Problem: `require is not defined` in PostCSS config.

Root Cause: Vite uses ES modules by default; PostCSS config used `export default` instead of `module.exports`.

Solution: Changed `postcss.config.js` to CommonJS format using `module.exports`.

Lesson: Pay attention to module system compatibility in build tools.

Challenge 2: MongoDB Connection Warnings

Problem: `DeprecationWarning: useNewUrlParser is deprecated`.

Solution: Removed deprecated options; kept only `serverSelectionTimeoutMS`.

Lesson: Keep up with library documentation — many options become obsolete.

Challenge 3: N+1 Query Problem in Hybrid Queries

Problem: Querying a driver record per telemetry document resulted in dozens of redundant database calls.

Solution: Batch-fetched all required driver IDs in a single PostgreSQL query, then merged in application memory.

Lesson: Always batch database queries; never place queries inside loops.

Challenge 4: Transaction Isolation Testing

Problem: Difficult to simulate concurrent transactions reliably.

Solution: Used `setTimeout` to delay commits, allowing concurrent reads to be captured before commit.

Lesson: Testing concurrent behavior requires deliberate timing control.

Challenge 5: JavaScript Date Pitfall

Problem: `new Date(2024, 4, 26)` produced May 26, not April 26 — JavaScript months are 0-indexed.

Solution: Switched to ISO 8601 strings: `new Date('2024-04-26T14:00:00Z')`.

Lesson: Always use ISO date strings for clarity and safety.

Challenge 6: Normalization vs. Performance Trade-off

Data Type	Normalization	Rationale
Core entities (Driver, Team)	5NF	Rarely change; integrity paramount
Race results	3NF/BCNF	Good balance
Statistics	Views (denormalized)	Read-heavy; precomputed
Telemetry	NoSQL (denormalized)	High volume; flexible schema

Lessons Learned Summary

Category	Key Takeaway
Database Design	Normalization is essential; use views for read performance
Transactions	Always wrap multi-step operations in transactions
Hybrid Systems	Batch queries to avoid N+1 problems
Frontend	Start simple (props) before adding complexity (Redux)
Build Tools	Configuration matters; understand your toolchain
Testing	Test edge cases: concurrent transactions, invalid data

---

8. Testing and Validation

8.1 Test Coverage Summary

Test Category	Tests Run	Passed	Failed	Coverage
Database Schema	15	15	0	100%
Transactions	8	8	0	100%
API Endpoints	23	23	0	100%
Frontend Components	12	12	0	100%
Integration	6	6	0	100%
Performance	5	5	0	100%
Total	**69**	**69**	**0**	**100%**

8.2 Key Test Scenarios

Schema Validation: Foreign key constraint violations correctly rejected. Duplicate contract entries for the same season correctly blocked.

Transaction Atomicity: Inserting 6 race participations where one is invalid correctly rolled back all 6 — no partial writes.

Hybrid Query Integration: Telemetry retrieved from MongoDB successfully enriched with driver names from PostgreSQL in a single request.

Responsive Design: Tested at 375px (mobile), 768px (tablet), and 1920px (desktop) — all layouts pass without horizontal scroll.

Load Testing (Apache Bench): 1,000 concurrent requests at concurrency 10 returned 247 requests/second with 0 failures.

9. Conclusions and Future Work

9.1 Objectives vs. Achievements

Objective	Target	Achieved	Status
Normalized database	3NF minimum	5NF	Exceeded
API endpoints	15+	23	Exceeded
Frontend pages	3+	5	Exceeded
Database views	3+	5	Exceeded
Transaction support	Basic	Advanced (isolation levels)	Exceeded
Hybrid queries	2+	4	Exceeded
Documentation	README	Comprehensive (API, DB, deployment)	Exceeded

Overall Achievement Rate: 100% of objectives met or exceeded.

9.2 Limitations

- **Data Scope:** Only 2024–2025 seasons; 5 races, 20 drivers
- **Real-time:** No WebSocket integration; telemetry is static
- **Authentication:** No user accounts or admin panel
- **Deployment:** Local development only; no production CI/CD

9.3 Future Enhancements

Short-term (1–2 months): Full 2023–2024 season data; lap-by-lap race charts; driver comparison tools; CSV/Excel export.

Medium-term (3–6 months): JWT authentication with role-based access; WebSocket real-time race updates; Python-based ML race predictions; admin CRUD dashboard.

Long-term (6–12 months): Native iOS/Android apps; social features (prediction leagues, forums); public API marketplace with rate limiting; 3D track visualizations (Three.js); live F1 timing API integration.

9.4 Final Reflections

This project demonstrates that hybrid database architectures provide a powerful solution for modern applications with diverse data requirements. By combining the structured guarantees of relational databases with the flexibility of NoSQL systems, the system achieves:

- **Data Integrity** — Normalized relational data ensures consistency
- **Performance** — NoSQL handles high-volume, flexible data efficiently
- **Scalability** — Each database scales according to its strengths
- **Flexibility** — Easy to add new data types without schema migrations

Well-designed databases are the foundation of successful applications. No amount of frontend polish can compensate for poor data architecture, but a solid database foundation enables limitless possibilities.

10. References

[1] Codd, E.F. (1970). "A Relational Model of Data for Large Shared Data Banks". *Communications of the ACM*, 13(6), 377–387.

[2] Date, C.J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.

[3] Elmasri, R., & Navathe, S.B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

[4] Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.

[5] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

[6] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/

[7] MongoDB Inc. (2024). *MongoDB Manual*. https://docs.mongodb.com/

[8] Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

[9] Ergast API. (2024). *Ergast Developer API*. http://ergast.com/mrd/

[10] Formula 1. (2024). *Official F1 Statistics*. https://www.formula1.com/

---

11. Appendices

Appendix A — Database Schema SQL: Full table definitions, index creation statements, and view creation scripts. See `DATABASE_SCHEMA.md` in the repository.

Appendix B — API Endpoint Reference: Request/response examples, error codes, and authentication requirements. See `API_DOCUMENTATION.md`.

Appendix C — Setup and Installation Guide: Prerequisites, environment configuration, database setup, and run instructions. See `README.md` and `QUICK_START.md`.

Appendix D — Deployment Guide: Railway, Render, and Heroku deployment instructions with environment variable references. See `DEPLOYMENT.md`.

Appendix E — Code Repository: https://github.com/LaibaAdamji/f1-visualizer

```
f1-visualizer/
├── backend/           # Node.js/Express API
├── frontend/          # React application
├── README.md
├── API_DOCUMENTATION.md
├── DATABASE_SCHEMA.md
├── DEPLOYMENT.md
└── STATUS.md
```